Habib University

Dhanani School of Science and Engineering

Computer Architecture

EE 371 / CS 330 / CE 321 (Fall 2025)

Homework 3 Solution

Total Marks: 100

Instructions:

1. This assignment should be done in pairs.
2. All questions should be answered in **black ink only**.
3. Print the assignment, scan the solution, and upload it on HU LMS before the deadline.

Grading Criteria:

1. Your assignments will be checked by instructor/TA.
2. You can also be asked to give a viva where you will be judged whether you understood the question yourself or not. If you are unable to answer correctly to the question you have attempted right, you may lose your marks.
3. Zero will be given if the assignment is found to be plagiarized.
4. Untidy work will result in reduction of your points.

Submission policy:

No submission will be accepted after the instructor releases the solution on HU LMS.

CLO Assessment:

This assignment assesses students for the following course learning outcomes.

Course Learning Outcomes		CLO Assessed
CLO 1	***Explain*** the role of ISA in modern processors and instruction encodings and assembly language programming	
CLO 2	***Explain*** the architecture and working of a single cycle processor	
CLO 3	***Design*** the architecture to mitigate issues of a pipelined processor	✓
CLO 4	***Analyze the*** performance of cache operations	

Question # 1 (Points: 12, 2+2+2+2+2+2, CLO - 3):

A 5-stage pipelined processor has the following stage latencies:

IF	ID	EX	MEM	WB
250ps	180ps	300ps	270ps	150ps

And also consider the inter-stage register delay to be 20ps.

Also, assume that instructions executed by the processor for a program-A are divided as follows:

ALU/Logic	Jump/Branch	Load	Store
40%	10%	25%	25%

i. Compute the clock cycle time for both pipelined and non-pipelined versions.

Non-pipelined processor

$$\begin{aligned} T_{\text{non-pipe}} &= \text{IF} + \text{ID} + \text{EX} + \text{MEM} + \text{WB} \\ &= 250 + 180 + 300 + 270 + 150 \\ &= 1150 \text{ ps}. \end{aligned}$$

So each instruction takes **1150 ps** in the non-pipelined design.

Pipelined processor

In a pipeline, the clock period must accommodate the longest stage plus the inter-stage register delay:

$$\begin{aligned} T_{\text{pipe}} &= \max(\text{IF}, \text{ID}, \text{EX}, \text{MEM}, \text{WB}) + t_{\text{reg}} \\ &= \max(250, 180, 300, 270, 150) + 20 \\ &= 300 + 20 = 320 \text{ ps}. \end{aligned}$$

So pipeline clock period is **320 ps**.

ii. Determine the total latency for a store instruction in both processors.

Note: A store instruction writes data to memory in the MEM stage and does *not* perform a register write-back in WB. Therefore the WB stage is not used for stores.

Non-pipelined store latency

Sum the stages a store actually uses: IF, ID, EX, MEM (no WB):

$$\begin{aligned} \text{Store}_{\text{non-pipe}} &= \text{IF} + \text{ID} + \text{EX} + \text{MEM} \\ &= 250 + 180 + 300 + 270 \\ &= 1000 \text{ ps}. \end{aligned}$$

So non-pipelined store latency is **1000 ps**.

Pipelined store latency

A store goes through 4 pipeline stages: IF, ID, EX, MEM. Each pipeline stage takes one clock period $T_{\text{pipe}} = 320$ ps, so:

$$\text{Store}_{\text{pipe}} = 4 \times T_{\text{pipe}} = 4 \times 320 = 1280 \text{ ps}.$$

So pipelined store latency is **1280 ps**.

iii. If 1200 instructions are executed with no hazards, calculate the speedup achieved.

We compute execution time for both designs and then their ratio.

Non-pipelined execution time

Each instruction takes $T_{\text{non-pipe}} = 1150$ ps, so:

$$T_{\text{non}} = 1200 \times 1150 = 1{,}380{,}000 \text{ ps}.$$

Pipelined execution time (no hazards)

For a pipeline with k stages, the total number of cycles to complete N instructions (steady pipeline, no hazards) is:

$$\text{cycles} = N + (k - 1).$$

Here $k = 5$. Thus cycles $= 1200 + (5 - 1) = 1204$ cycles. Each cycle takes $T_{\text{pipe}} = 320$ ps, hence:

$$T_{\text{pipe\_total}} = 1204 \times 320 = 385{,}280 \text{ ps}.$$

Speedup

$$\text{Speedup} = \frac{T_{\text{non}}}{T_{\text{pipe\_total}}} = \frac{1{,}380{,}000}{385{,}280} \approx 3.584 \approx \mathbf{3.58}.$$

iv. Now assume a stall occurs every five instructions. What is the new speedup?

A stall every five instructions means we insert one bubble (one extra cycle) per 5 instructions.

Number of stalls for 1200 instructions

$$\text{stalls} = \frac{1200}{5} = 240.$$

(Assume this divides evenly.)

New pipeline cycle count

Previously, without stalls, cycles $= 1204$. Adding 240 stall cycles:

$$\text{cycles}_{\text{stalled}} = 1204 + 240 = 1444.$$

Execution time with stalls

$$T_{\text{stalled}} = 1444 \times 320 = 462{,}080 \text{ ps}.$$

New speedup

$$\text{Speedup}_{\text{stalled}} = \frac{1{,}380{,}000}{462{,}080} \approx 2.987 \approx \mathbf{2.99}.$$

v. Suggest a possible architectural modification to reduce overall cycle time without changing ISA behavior. Calculate the new pipeline clock cycle time.

Suggested modification

Split the longest stage (EX, 300 ps) into two smaller stages (say EX1 and EX2) so that no single stage is longer than the next-largest stage (MEM = 270 ps). This does not change ISA semantics; it only adds another pipeline register and splits EX into two pipeline stages.

Re-evaluate stage latencies after splitting EX

If EX is split into two equal parts:

$$\text{EX1} \approx 150 \text{ ps}, \quad \text{EX2} \approx 150 \text{ ps}.$$

The stage latencies become:

$$250,\ 180,\ 150,\ 150,\ 270,\ 150.$$

The new maximum stage latency is 270 ps (MEM).

New pipeline clock period

Add the register overhead:

$$T_{\text{new\_pipe}} = 270 + 20 = 290 \text{ ps}.$$

So with this modification, the pipeline clock reduces from 320 ps to **290 ps**.

(Optional) Note: Splitting EX increases pipeline depth (from 5 to 6), which affects the pipeline filling/emptying overhead: cycles $= N + (k' - 1)$ where $k' = 6$. For large N the small change in prologue/epilogue cycles is negligible relative to the reduction in clock period.

vi. Assuming all stages are fully utilized, calculate the utilization of the instruction memory and data memory over 1000 instruction executions.

Instruction memory utilization

Every instruction fetch uses instruction memory once (in IF). Thus, during 1000 instruction executions:

$$\text{Instruction memory accesses} = 1000.$$

Instruction memory utilization = **100%**.

Data memory utilization

Data memory is accessed by loads and stores only. From the instruction mix:

$$\text{Loads} = 25\%, \quad \text{Stores} = 25\% \quad \Rightarrow \quad 50\% \text{ access data memory.}$$

For 1000 instructions:

$$\text{Data memory accesses} = 0.50 \times 1000 = 500.$$

Thus data memory utilization = **50%**.

Question # 2 (Points: 10, 4+4+2, CLO - 3):

In a pipelined CPU, the multiply instruction (mul) requires two cycles in the EX stage (EXa and EXb)

i. Explain why this situation causes a structural hazard in the pipeline. Use a pipeline timing chart for a sequence containing two consecutive mul instructions to illustrate the conflict. (Assume there is no stall unit.)

Given code sequence:

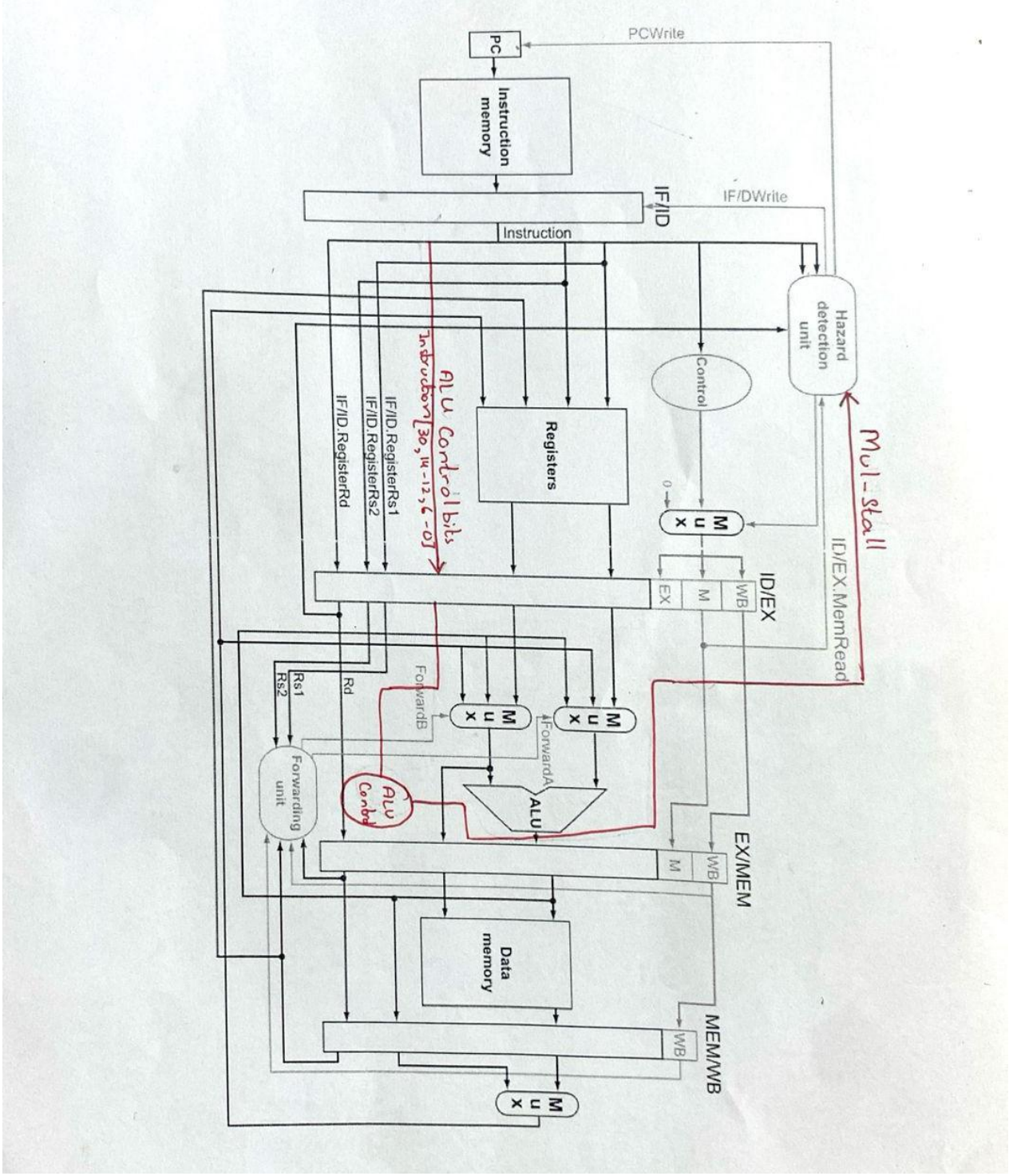
```
mul x1, x2, x3
mul x5, x4, x6
add x8, x2, x1
```

Pipeline timing chart:

Cycle	1	2	3	4	5	6	7
`mul x1,x2,x3`	IF	ID	EXa	EXb	M	WB	
`mul x5,x4,x6`		IF	ID	EXa	EXb	M	WB
`add x8,x2,x1`			IF	ID	EXa	M	WB

Observation: A structural hazard occurs at cycle 4, where both `mul` instructions need the EX unit simultaneously (`EXa` of the second overlaps with `EXb` of the first). In cycle 5, the second `mul` and the `add` instruction also overlap in the EX stage, and their `M` and `WB` stages align, further showing resource contention.

ii. Clearly highlight or annotate the modifications needed to support a two-cycle mul instruction using Figure 4.58 from the textbook. Indicate any new control signals, hardware components, or changes to existing control logic that prevent this hazard.



To prevent the structural hazard caused by consecutive mul instructions, a new signal MulStall (or EX busy) is added from the EX stage to the Hazard Detection Unit (HDU) in the ID stage. The modified control flow is shown conceptually in Figure 4.58 of the textbook.

- New Signal:

- MulStall is asserted by the ALU Control in the EX stage whenever a multiply instruction occupies both EXa and EXb.
- This signal is sent to the HDU to indicate that the EX hardware is busy.

- Hazard Detection Logic Update:

  The HDU now stalls the pipeline whenever the MulStall signal is active or a load-use hazard is detected. The updated stall condition is:

stall = (ID/EX.MemRead ∧ ((ID/EX.RegisterRd = IF/ID.RegisterRs1) ∨ (ID/EX.RegisterRd = IF/ID.Register2))) ∨ MulStall

When stall = 1, the HDU performs the following actions:
- PCWrite ← 0 (freeze Program Counter)
- IF/ID.Write ← 0 (hold current instruction in Decode)
- ID/EX.control ← 0 (insert a NOP bubble into the pipeline)

Effect on Pipeline: The next instruction following a mul is stalled in the ID stage until the multiplier finishes execution (i.e., EX busy deasserts). This prevents both EXa and EXb from being occupied simultaneously.

Summary: Adding one feedback signal (MulStall) and slightly extending the hazard detection logic ensures that the pipeline stalls automatically whenever a multi-cycle mul is in progress. No additional forwarding paths or ALU hardware are required

iii. Briefly describe how your proposed modifications resolve the hazard.

Resolution of the Hazard:

Instruction	1	2	3	4	5	6	7	8	9
`mul x1, x2, x3`	IF	ID	EXa	EXb	M	WB			
`mul becomes nop`		IF	ID	✡	✡	✡	✡	✡	
`mul x5, x4, x6`			IF	ID	EXa	EXb	M	WB	
`add becomes nop`				IF	ID	✡	✡	✡	✡
`add x8, x2, x1`					IF	ID	EXa	M	WB

Explanation: In this modified pipeline, the `MulStall` signal generated by the ALU Control unit activates the hazard detection unit whenever a `mul` instruction is in progress (occupying both `EXa` and `EXb`). The HDU stalls the subsequent instruction in the Decode (`ID`) stage, as shown by the stall bubbles (✡). This ensures that no new instruction enters the `EX` stage until the first `mul` finishes execution.

Question # 3 (Points: 8, 5+3, CLO - 3):

A processor uses a 2-bit saturating counter branch predictor with the following encoding:

Binary	**Meaning**
00	Strongly Not Taken
01	Weakly Not Taken
10	Weakly Taken
11	Strongly Taken

Initially, the Branch History Table (BHT) contains the entries of the states of four branches A, B, C, and D as:

- A = 10
- B = 00
- C = 11,
- D = 01

During program execution, the following sequence of branch executions and actual outcomes occurs:

Order	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**
Branch	A	B	A	C	A	D	C	A	B	10
Actual Outcome	Taken	Not Taken	Taken	Taken	Not Taken	Not Taken	Taken	Taken	Not Taken	Not Taken

i. Simulate the branch predictor operation for this sequence. For each branch execution, show:

- the initial state
- the prediction made
- whether it was correct
- the updated state.

Order	Branch	Outcome	Initial State	Prediction	Correct?	Updated State
1	A	Taken	10	Taken	Yes	11
2	B	Not Taken	00	Not Taken	Yes	00
3	A	Taken	11	Taken	Yes	11
4	C	Taken	11	Taken	Yes	10
5	A	Not Taken	11	Taken	No	10
6	D	Not Taken	01	Not Taken	Yes	10
7	C	Taken	10	Taken	Yes	01
8	A	Taken	10	Taken	Yes	11
9	B	Not Taken	00	Not Taken	Yes	00
10	C	Not Taken	01	Taken	No	00

ii. Count the total mispredictions and compute the misprediction rate.

From the table:

$$\text{Total mispredictions} = 2$$

$$\text{Misprediction Rate} = \frac{2}{10} = 0.20 = 20\%$$

Question # 4 (Points: 24, (2+5+3+2)x2, CLO - 3):

Define new instructions for RISC-V

i. maxadd rd, rs1, rs2, rs3 // Reg[rd] = max(Reg[rs1], Reg[rs2]) + Reg[rs3])

a. Suggest a suitable instruction format (using existing R, I, S, or new type).

The instruction performs:

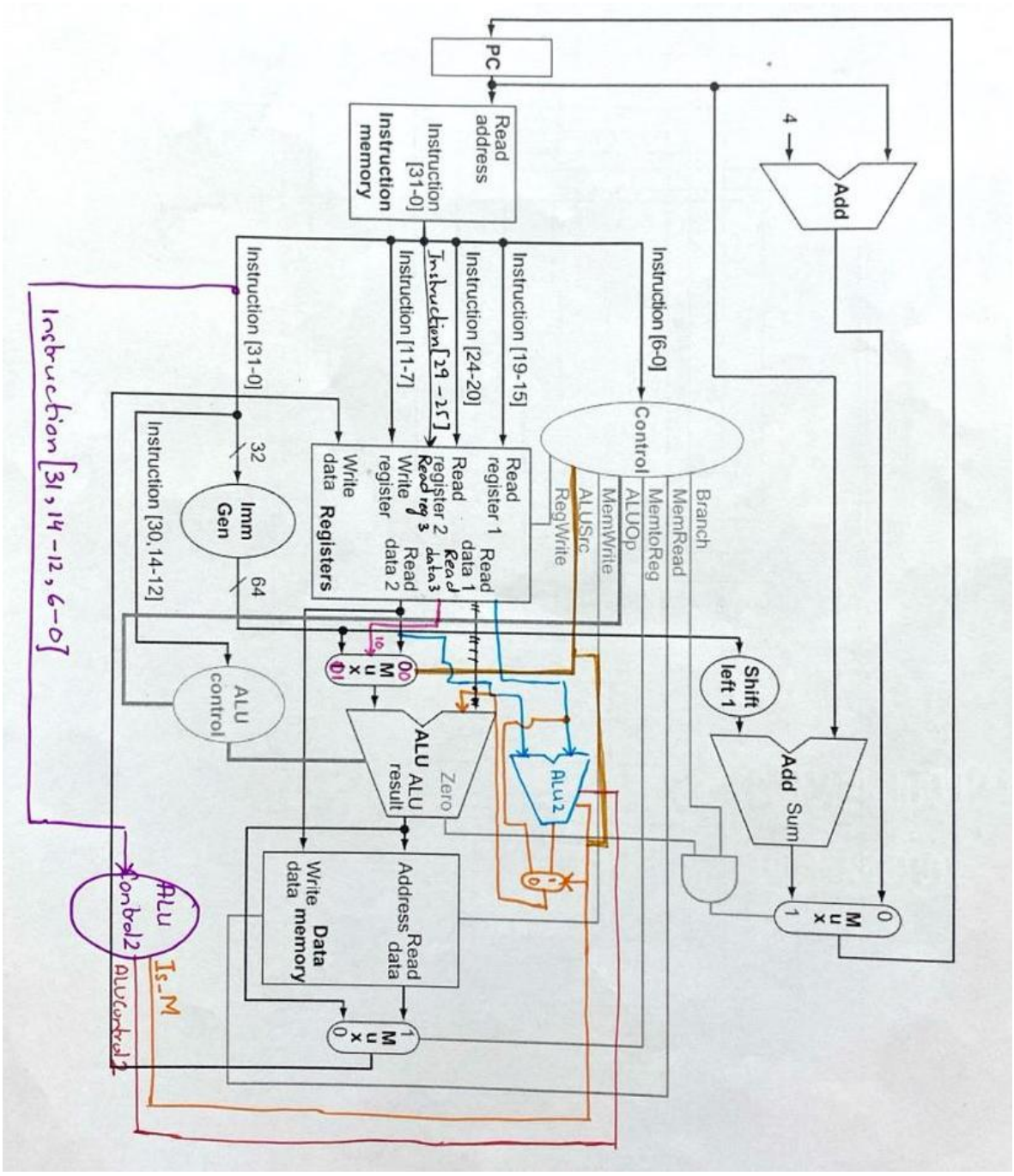
$$\text{Reg[rd]} = \max(\text{Reg[rs1]}, \text{Reg[rs2]}) + \text{Reg[rs3]}$$

Since this operation requires three source registers, it cannot be encoded using the standard RISC-V `R`, `I`, or `S` formats. We define a new **Type-M** format that adds one additional source field (`rs3`) while maintaining a fixed 32-bit width.

[31:30]	[29:25]	[24:20]	[19:15]	[14:12]	[11:7]	[6:0]
10	rs3	rs2	rs1	funct3	rd	opcode

This `Type-M` format supports three register operands and can be placed within the RISC-V without breaking compatibility.

b. Describe datapath modifications required for this instruction in a single-cycle CPU. You can use a printed version of the figures in the book over which you can draw your suggested modifications.



c. What new control signals must be generated?

To implement the new instruction, the following additional control signals are introduced:

1. `Is_M`: Identifies when the current instruction belongs to the new Type-M format.

$$\texttt{Is\_M} = \begin{cases} 1, & \text{if instruction is of Type-M} \\ 0, & \text{otherwise.} \end{cases}$$

   This signal enables the additional datapath logic (second ALU and extended multiplexers) used by all Type-M operations.
2. `ALUControl2`: Control bits for the second ALU. When asserted, this signal enables the additional ALU in the EX stage that compares the two registers (`rs1` and `rs2`) and outputs the maximum of the two.
3. `Mux_ALU_BSel`: A 2-bit selector for the second ALU input.

$$\texttt{Mux\_ALU\_BSel} = \{\texttt{Is}_M \texttt{Is}_{M\,\texttt{Is}_{M\,\texttt{Is}_M.\texttt{ALUSrc}}}\}$$

$$00 : \texttt{rs2}$$
$$01 : \text{Immediate value}$$
$$10 : \texttt{rs3}$$

These additions allow the datapath to route the third source register (`rs3`) to the ALU and perform the `max(Reg[rs1], Reg[rs2]) + Reg[rs3]` operation without altering existing instruction behavior.

d. Identify any new forwarding paths needed for a pipelined version.

The same forwarding mechanism is reused for the `maxadd` instruction. However, the forwarding unit must now also check data dependencies for the additional source register (`rs3`). New multiplexers are added to the ALU inputs to allow forwarded values for `rs3`, and one extra forwarding control signal is generated. This ensures that any of the three operands (`rs1`, `rs2`, or `rs3`) can receive the most recent value from the `EX/MEM` or `MEM/WB` pipeline registers without introducing stalls.

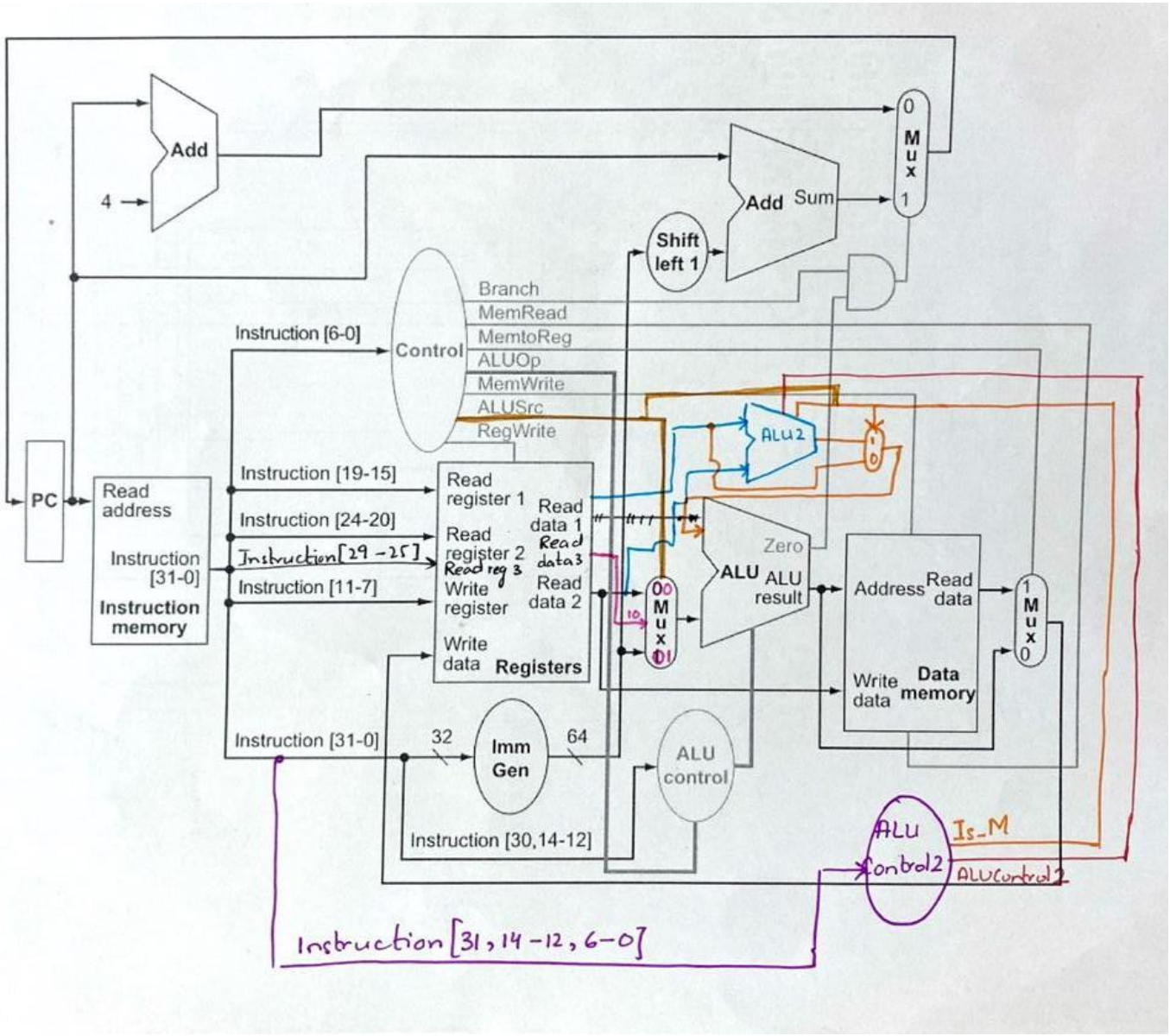
ii. cmpadd rd, rs1, rs2, rs3 // if (Reg[rs1] > Reg[rs2]) Reg[rd] = Reg[rs3] + 1; else Reg[rd]=Reg[rs3];

a. Suggest a suitable instruction format (using existing R, I, S, or new type).

Like the `maxadd` instruction, it requires three source registers and thus uses the same custom **Type-M** format.

[31:30]	[29:25]	[24:20]	[19:15]	[14:12]	[11:7]	[6:0]
10	rs3	rs2	rs1	funct3	rd	opcode

b. Describe datapath modifications required for this instruction in a single-cycle CPU. You can use a printed version of the figures in the book over which you can draw your suggested modifications.


Add
4
Add
Sum
Shift left 1
0
M
u
x
1
Branch
MemRead
MemtoReg
ALUOp
MemWrite
ALUSrc
RegWrite
Control
Instruction [6-0]
PC
Read address
Instruction [31-0]
Instruction memory
Instruction [19-15]
Instruction [24-20]
Instruction [29-25]
Instruction [11-7]
Read register 1
Read register 2
Write register
Write data
Read data 1
Read data 2
Registers
ALU2
Zero
ALU
ALU result
Address
Read data
Write data
Data memory
Instruction [31-0]
32
Imm Gen
64
ALU control
Instruction [30,14-12]
ALU Control2
Is_M
ALUControl2
Instruction [31,14-12,6-0]

c. What new control signals must be generated?

No additional control signals are introduced beyond those used for `maxadd`. Only the decoding and ALU operation differ.

1. `isCmpAdd`: Identifies when the current instruction is a `cmpadd`.
2. `ALUControl2`: Controls the second ALU in the EX stage. For this instruction, it compares `Reg[rs1]` and `Reg[rs2]` and outputs 1 if `rs1 > rs2`, otherwise 0. This result determines whether 1 is added to `Reg[rs3]` in the main ALU.

Thus, the `cmpadd` instruction reuses the same extended datapath, with the second ALU performing a comparison instead of a maximum operation.

d. Identify any new forwarding paths needed for a pipelined version.

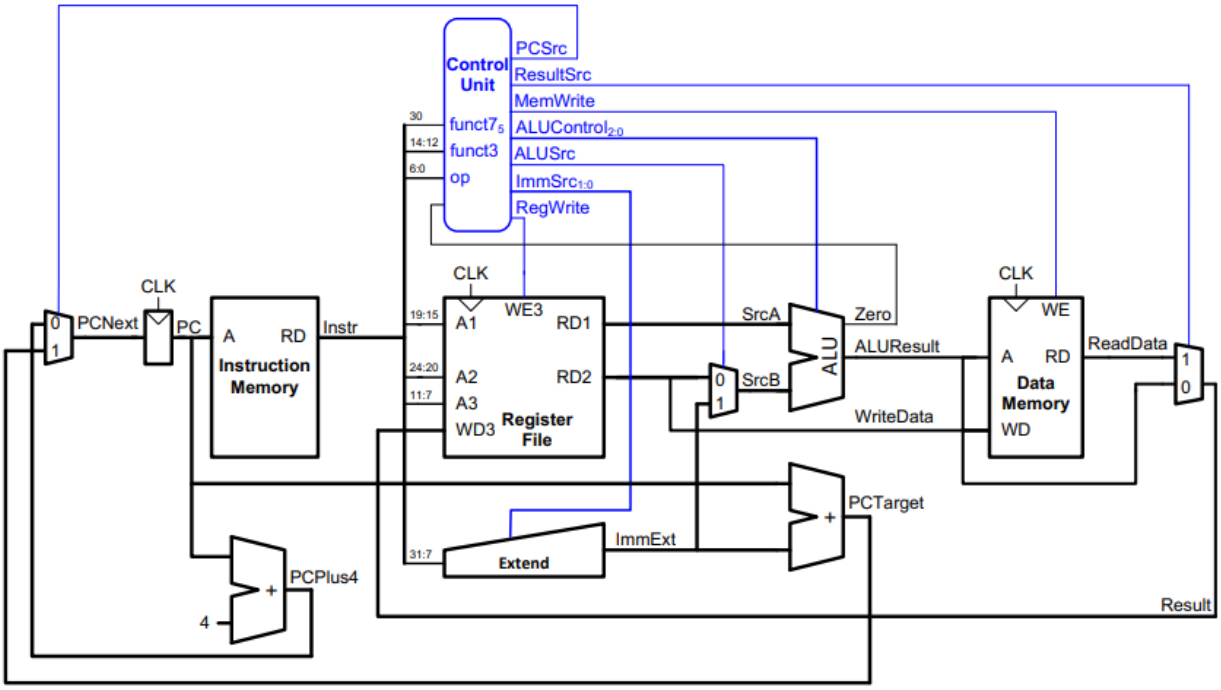
The same forwarding mechanism is reused for the `cmpadd` instruction. However, the forwarding unit must now also check data dependencies for the additional source register (`rs3`). New multiplexers are added to the ALU inputs to allow forwarded values for `rs3`, and one extra forwarding control signal is generated. This ensures that any of the three operands (`rs1`, `rs2`, or `rs3`) can receive the most recent value from the `EX/MEM` or `MEM/WB` pipeline registers without introducing stalls.

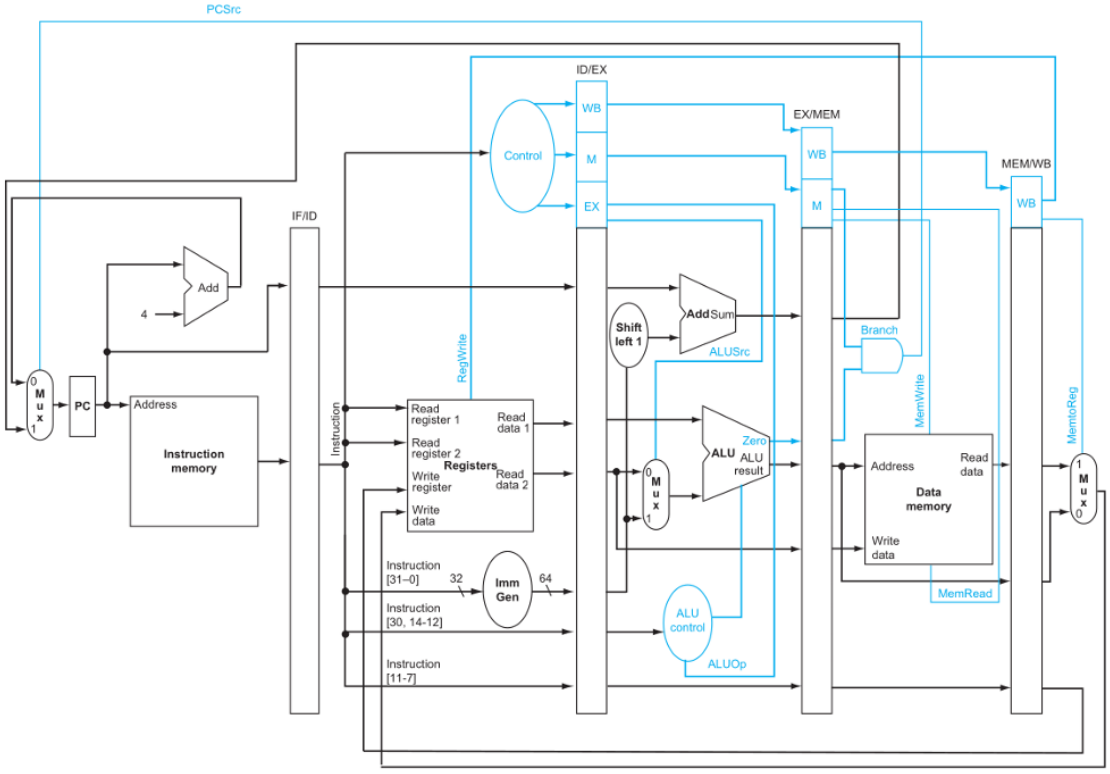
Question # 5 (Points: 15, 1.5x10, CLO - 3):

Suppose one of the following control signals in a single-cycle RISC-V processor is stuck-at-1 (always logic 1).

Which instruction(s) would malfunction, what would be the issues observed and why?

Refer to the extended single-cycle datapath discussed in class.





Instruct	RegWrite	ImmSrc	ALUSrc	MemWrite	ResultSrc	Branch	ALUOp
`lw`	1	00	1	0	1	0	00
`sw`	0	01	1	1	X	0	00
R-type	1	XX	0	0	0	0	10
`beq`	0	10	0	0	X	1	01

i. RegWrite

Instructions with no destination register would malfunction as RegWrite = 1 indicates that the ALU result has to be written in a destination register and those instructions do not have one.

These include S-Type, SB-Type and those instructions which do not write an rd.

ii. MemRead

MemRead is 1 only for load instructions. All the other instructions would malfunction if it gets stuck at 1 because they do not need to read data from memory.

These include R-type, S-Type, SB-type, I-type (except load), U-type, and UJ-type.

iii. MemWrite

MemWrite is 1 for instructions that need to write data to memory instead of registers. All the instructions that write to a destination register instead of memory would malfunction in this case.

These include R-Type, I-Type, SB-type, U-type, and UJ-type.

iv. Branch

If Branch is stuck at 1, it would make all the instructions as branch instructions even when they are not. The PC will use branch target instead of PC+4 for all the instructions which will result in incorrect fetching.

This would result in malfunction of all the instructions except branch instructions.

v. ALUSrc

ALUSrc gives 1 for instructions whose second operand (readdata2) of ALU comes from immediate and not from rs2. If it is stuck at 1, all the instructions that uses rs2 instead of immediate will malfunction.

These include R-type, S-type, SB-type.

vi. $ResultSrc_0$

ResultSrc gives 1 when result has to come from memory and 0 when result will be taken from ALU. If it is stuck at 1, all the instructions that take result from ALU would malfunction.

These include R-type, I-type (except load).

vii. $ImmSrc_1$

ImmSrc is a 2 bit control signal which tells which format the immediate should be extended into. It typically follows the mapping: 00=I, 01=S, 10=B, 11=U. If it's MSB is stuck at 1, then I and S type instructions will malfunction and will be fed incorrectly extended immediate.

viii. PCSrc

PCSrc selects between PC+4 and branch target address. If it is stuck at 1, it will always select branch target address to fetch next instruction, which will result in malfunctioning of branch not taken and all other instructions except branch taken.

ix. $ALUOp_1$

ALUOp is a 2 bit signal that determines the operation ALU will be performing. It is mapped as 00 = add, 01 = sub, 10 = R-type. If $ALUOp_1$ is stuck at 1, load, store, and branch instructions will malfunction.

x. $ALUOp_0$

If $ALUOp_0$ is stuck at 1, R-type and load and store instructions which require add will malfunction.

Question # 6 (Points: 13, 3+4+3+3, CLO - 3):

The pipelined RISC-V processor executes the following code segment:

```
addi x1, x0, 5      # x1 = 5
addi x2, x0, 10     # x2 = 10
add x3, x1, x2      # x3 = 15
sub x4, x3, x1      # x4 = 10
lw x5, 0(x4)        # load word from memory address in x4
add x6, x5, x1      # x6 = x5 + x1
sw x6, 4(x4)        # store word from x6 to memory[x4 + 4]
add x7, x6, x3      # x7 = x6 + x3
```

Assume the following initial conditions:

- All registers are initialized to 0 except x0 (which is always 0).
- Memory location Mem[10] = 40.

i. Identify all data hazards present in the above sequence (specify the dependent registers and the type of hazard).

1. add x3 x1 x2: data hazard – MEM/WB.RegisterRd = ID/EX.RegisterRs1 = x1
2. sub x4 x3 x1: data hazard – EX/MEM.RegisterRd = ID/EX.RegisterRs1 = x3
3. lw x5 0(x4): data hazard - EX/MEM.RegisterRd = ID/EX.RegisterRs1 = x4
4. add x6 x5 x1: data hazard - EX/MEM.RegisterRd = ID/EX.RegisterRs1 = x5
5. sw x6 4(x4): data hazard - EX/MEM.RegisterRd = ID/EX.RegisterRs2 = x6
6. add x7 x6 x3: data hazard – MEM/WB.RegisterRd = ID/EX.RegisterRs1 = x6

ii. Assuming there is no forwarding and no stall detection, complete a table showing the value of each register (x1–x7) after each instruction finishes execution. Clearly indicate where incorrect values would appear due to hazards.

X1	5	correct
X2	10	correct
X3	0	incorrect
X4	-5	incorrect
X5	Mem[0]	incorrect
X6	5	incorrect
sw x6 4(x4)	Mem[-1] = 0	incorrect
X7	0	incorrect

iii. Describe how forwarding and hazard detection unit could be used to ensure correct execution of this program. Mention which hazards are resolved by forwarding and which require stalls.

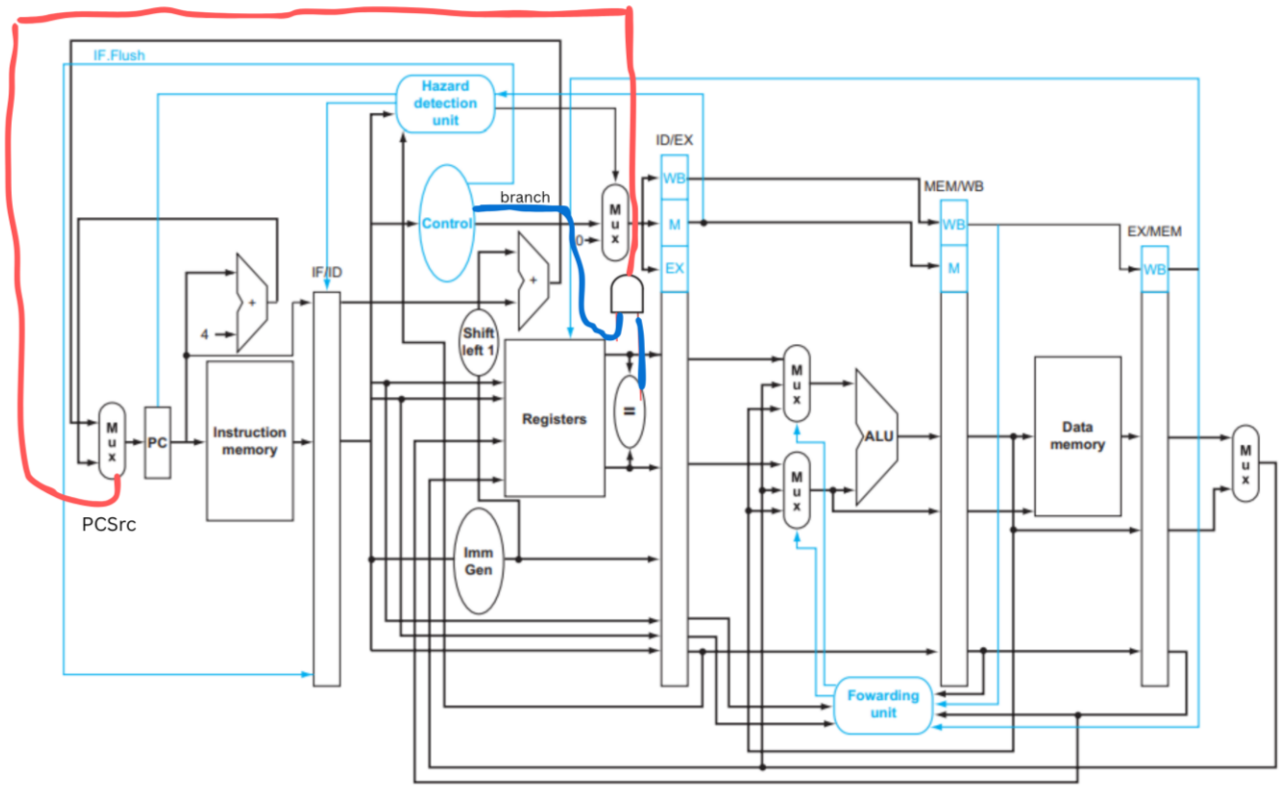
1. add x3 x1 x2: data hazard – resolved by forwarding from MEM/WB.Register
2. sub x4 x3 x1: data hazard – resolved by forwarding from EX/MEM.Register
3. lw x5 0(x4): data hazard – resolved by forwarding from EX/MEM.Register
4. add x6 x5 x1: data hazard – resolved by a stall and forwarding from MEM/WB.Register
5. sw x6 4(x4): data hazard – resolved by forwarding from EX/MEM.Register
6. add x7 x6 x3: data hazard – resolved by forwarding from MEM/WB.Register

iv. Redraw the pipeline diagram assuming full forwarding and hazard detection. Clearly label where stalls (if any) occur.

	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13
addi x1 x0 5	IF	ID	EX	MEM	WB								
addi x2 x0 10		IF	ID	EX	MEM	WB							
add x3 x1 x2			IF	ID	EX	MEM	WB						
sub x4 x3 x1				IF	ID	EX	MEM	WB					
lw x5 0(x4)					IF	ID	EX	MEM	WB				
NOP													
add x6 x5 x1							IF	ID	EX	MEM	WB		
sw x6 4(x4)								IF	ID	EX	MEM	WB	
add x7 x6 x3									IF	ID	EX	MEM	WB

Question # 7 (Points: 10, CLO - 3):

The pipelined processor's performance might be better if branches take place during the Decode stage rather than the Execute stage. Show how to modify the pipelined processor to move the branch logic to the Decode stage. How do the stall, flush, and forwarding signals change? You can use a printed version of the figures in the book over which you can draw your suggested modifications.



There is a comparator introduced in the ID stage whose output will become one of the inputs of the AND gate and the branch control signal will go into the AND gate as the second input. If the AND output is 1, PCSrc signal is high and it makes the PC jump to branch target address.

Because the value in a branch comparison is needed during ID but may be produced later in time, it is possible that a data hazard can occur and a stall will be needed. For example, if an ALU instruction immediately preceding a branch produces the operand for the test in the conditional branch, a stall will be required, since the EX stage for the ALU instruction will occur after the ID cycle of the branch. By extension, if a load is immediately followed by a conditional branch that depends on the load result, two stall cycles will be needed, as the result from the load appears at the end of the MEM cycle but is needed at the beginning of ID for the branch. To cater that, a new forwarding logic can be implemented, where the bypassed source operands of a branch can come from either the EX/MEM or MEM/WB pipeline registers.

Question # 8 (Points: 8, 2+2+2+2, CLO - 3):

A 64-bit RISC-V pipeline is being redesigned to improve performance by modifying its Execute (EX) stage.
The pipeline clock period is determined by the longest stage delay, including a 20 ps delay for each inter-stage pipeline register.
Each ALU has a propagation delay of 150 ps, and the EX stage dominates all other stage latencies in the pipeline.

Three configurations are considered:

- Configuration A: Single ALU in the EX stage (baseline design).
- Configuration B: Two ALUs cascaded within the same EX stage (for multi-operand operations).
- Configuration C: EX stage split into two pipeline stages, EX1 and EX2, each containing one ALU.

i. Compute the pipeline clock cycle time for Configurations A, B, and C.

Configuration A:
Clock cycle time = 150 + 20 = 170 ps

Configuration B:
Clock Cycle time = 150 + 150 + 20 = 320 ps

Configuration C:
Clock Cycle time = 150 + 20 = 170 ps

ii. Assuming Register 1, Register 2, and Register 3 represent 64-bit source registers (rs1, rs2, rs3), and ignoring control signals, compute the width (in bits) of the inter-stage pipeline registers for:
- Configuration B: ID/EXE and EXE/MEM
- Configuration C: ID/EXE1, EXE1/EXE2, and EXE2/MEM

Configuration B:

ID/EXE → 3 * 64 = 192 bits (rs1,rs2,rs3)

EXE/MEM → minimum: 64 bits (ALU result)
maximum: 64 + 64 = 128 bits (if instruction is store we would need to pass rs2 data as well)

Configuration C:

EX1 contains ALU1 which takes rs1, rs2 and gives an intermediate result of 64 bits, call it x.
EX2 contains ALU2 which takes x and rs3 and produces final 64-bit result.

ID/EXE1 → 3 * 64 = 192 bits (rs1,rs2,rs3)

EXE1/EXE2 → x + rs3: 64 + 64 = 128 bits

EXE2/MEM → minimum: 64 bits (final result)
maximum: 64 + 64 = 128 bits (if instruction is store we would need to pass store data as well)

iii. Assume both ALUs perform the same operation. Identify and name the control signals required for the ALU stage(s) in Configurations B and C.

Configuration B:

1. ALUOp (1 ALUOp since both perform same operation)

2. ALUSrc1 (for ALU1) :
   - ALU1_selA → selects between rs1, forwarded value from EXE/MEM, forwarded from MEM/WB
   - ALU1_selB → selects between rs2, immediate, forwarded value

3. ALUSrc2 (for ALU2):
   - ALU2_selA → selects between ALU1 output, forwarded operand
   - ALU2_selB → selects between rs3, immediate, forwarded value

4. ALU_enable → enable or bypass an ALU

5. ForwardA1 / ForwardB1 → chooses source for ALU1 inputs

6. ForwardA2 / ForwardB2 → chooses source for ALU2 inputs

Configuration C:

EX1 (ALU1):

1. ALUOp (1 aluop for both EX1 and EX2 since operation is identical)

2. ALU1_selA

3. ALU1_selB

4. ForwardA1 / ForwardB1 → forwarding from EXE2/MEM or MEM/WB

EX2 (ALU2):

1. ALUOp

2. ALU2_selA

3. ALU2_selB

5. ForwardA2 / ForwardB2 → forwarding from EXE2/MEM or MEM/WB or EXE1/EXE2

iv. For Configuration B, write the hazard detection and forwarding conditions needed to handle data dependencies between consecutive ALU instructions.
Use pipeline register notation (e.g., EX/MEM.RegisterRd = ID/EX.RegisterRs1).

Forwarding:

ForwardA1/ForwardB1:
(00= ID/EXE register value, 10 = EXE/MEM.ALUResult, 01 = MEM/WB.WriteBackData)

ForwardA2/ForwardB2:
(00 = ALU1 output or ID/EXE.rs3, 10 = EXE/MEM.ALUResult, 01=MEM/WB.WriteBackData)

1. **Forward from EXE/MEM to ALU input A (rs1):**

if (EXE/MEM.RegWrite == 1 AND EXE/MEM.RegisterRd != 0) AND
(EXE/MEM.RegisterRd == ID/EXE.RegisterRs1)

then ForwardA = 10

else if (MEM/WB.RegWrite == 1 AND MEM/WB.RegisterRd != 0) AND
(MEM/WB.RegisterRd == ID/EXE.RegisterRs1)

then ForwardA = 01

else ForwardA = 00

2. **Forward from EXE/MEM to ALU input B (rs2):**

if (EXE/MEM.RegWrite == 1 AND EXE/MEM.RegisterRd != 0) AND
(EXE/MEM.RegisterRd == ID/EXE.RegisterRs2)

then ForwardB = 10

else if (MEM/WB.RegWrite == 1 AND MEM/WB.RegisterRd != 0) AND
(MEM/WB.RegisterRd == ID/EXE.RegisterRs2)

then ForwardB = 01

else ForwardB = 00

3. Forwarding for rs3:

```
if (EXE/MEM.RegWrite == 1 AND EXE/MEM.RegisterRd != 0 ) AND
( EXE/MEM.RegisterRd == ID/EXE.RegisterRs3 ) then

ForwardC = 10

else if (MEM/WB.RegWrite == 1 AND  MEM/WB.RegisterRd != 0 ) AND
( MEM/WB.RegisterRd == ID/EXE.RegisterRs3 ) then

ForwardC = 01

else
ForwardC = 00
```

Hazard Detection:

```
if ( ID/EXE.MemRead == 1 ) AND
  ( ( ID/EXE.RegisterRd == IF/ID.RegisterRs1 ) OR
    ( ID/EXE.RegisterRd == IF/ID.RegisterRs2 ) OR
    ( ID/EXE.RegisterRd == IF/ID.RegisterRs3 ) )
then
```

- Insert one bubble
- Freeze PC and IF/ID (do not fetch next instruction)
- Insert a bubble (NOP) into ID/EXE (clear control signals for that stage)
- After one cycle the load reaches MEM and its data can be forwarded